



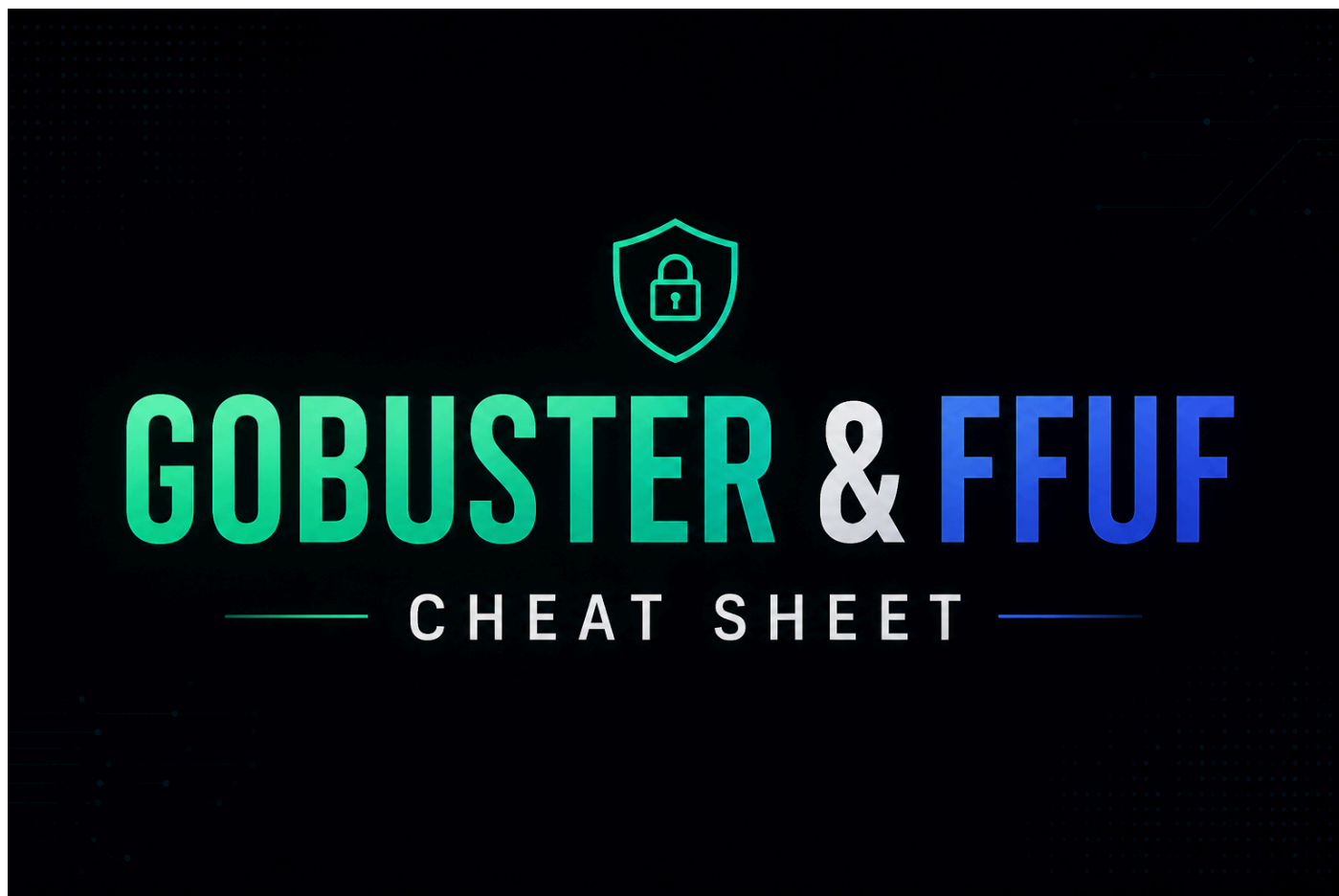
Содержание



Присоединяйтесь к нашему сообществу, чтобы начать свой путь в сфере кибербезопасности уже сегодня!

[Узнать больше](#)

Полное руководство по Gobuster и ffuf



1. Что такое Gobuster и Ffuf? (И когда использовать каждый из них)

Вы составили карту целевого веб-приложения, перехватили часть трафика Burp Suite и нашли несколько видимых страниц. Но за ними скрываются панели

Взламывающие Петли



пий, промежуточные среды, конечные
рые не ведет ни одна ссылка на странице.
веб-фаззинга, а именно Gobuster и ffuf —
два инструмента, которые должны быть у каждого пентестера.

Оба инструмента написаны на Go, оба работают невероятно быстро и отправляют тысячи HTTP-запросов с разными путями или значениями, анализируя полученные ответы. Ключевое различие заключается в охвате и гибкости.

Пожиратель

ffuf

Основная цель	Поиск по каталогам/файлам, DNS, виртуальным хостам — оптимизировано для поиска методом перебора	Универсальный веб-фаззер — фаззер для всего: URL, заголовков, параметров, данных POST, файлов cookie
Ключевое слово/ позиция	Автоматически добавляется к URL-адресу (заполнитель не требуется)	Использует ключевое слово FUZZ — его можно разместить в любом месте запроса
Кривая обучения	Проще — меньше флагов, больше мнений	Более мощный, но требует понимания принципов работы фильтров для получения чистого результата
Фильтрация	Базовая фильтрация кода состояния	Расширенный — фильтрация по статусу, размеру, словам, строкам, регулярным выражениям, времени

Взламывающие Петли



Да — флаги `-recursion` и `-recursion-depth`

Несколько списков слов	Один список слов за прогон	Несколько списков слов для одновременного использования с именованными ключевыми словами
Лучше всего подходит для	Быстрая первичная разведка, перебор DNS, обнаружение виртуальных хостов	Фаззинг параметров, POST-данные, обход аутентификации, тестирование API, расширенная фильтрация

Эмпирическое правило

Используйте `Gobuster` для быстрой первичной проверки — он быстрее в наборе и отлично подходит для наиболее распространенных случаев (перебора каталогов). Переходите на `ffuf`, когда вам нужно перебрать параметры, заголовки, данные POST или что-то еще, для чего нужно разместить заполнитель не в конце URL-пути.

2. Установка и настройка

Kali Linux — оба инструмента установлены по умолчанию

Взламывающие Петли



uster ffuf -y

```
gobuster version
```

```
ffuf -V
```

Ручная установка (любая версия Linux / macOS)

```
# Gobuster - ИЗ Go binary
```

```
перейти к установке github.com/OJ/gobuster/v3@latest
```

```
# ffuf - ИЗ Go binary
```

```
перейдите к установке github.com/ffuf/ffuf/v2@latest
```

```
# ffuf - ИЗ ГОТОВОГО двоичного файла (Linux x64)
```

```
wget
```

```
https://github.com/ffuf/ffuf/releases/latest/download/ffuf\_Linux\_amd64.tar.gz
```

```
tar -xvf ffuf_Linux_amd64.tar.gz
```

```
chmod + x ffuf && sudo mv ffuf /usr/ local/bin/
```

Windows

```
# Скачать gobuster С: https://github.com/OJ/gobuster/releases
```

```
# Скачать ffuf С: https://github.com/ffuf/ffuf/releases
```

```
# Распаковать ZIP и запустить из распакованной папки:
```

```
gobuster.exe -help
```

```
ffuf.exe -h
```

Установка SecLists (основных списков слов)

```
# Kali Linux
```

```
sudo apt install seclists -y
```

```
# Установлено в: /usr/share/seclists/
```

3. Списки слов: основа эффективного фаззинга

Ни один из инструментов не находит директории по волшебству. Они отправляют каждую запись из списка слов в виде запроса и отмечают интересные ответы. Качество вашего списка слов — самый важный фактор, влияющий на то, что вы найдете.

Рекомендуемые списки слов в зависимости от сценария использования

Пример использования	Список Слов	Путь (Кали)
Обнаружение директорий/файлов (быстрое)	common.txt	/usr/share/seclists/Discovery/Web-Content/common.txt
Обнаружение директорий/файлов (тщательное)	directory-list-2.3-medium.txt	/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
Перебор каталогов (большой)	raft-large-directories.txt	/usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt
Обнаружение файлов (с расширениями)	raft-large-files.txt	/usr/share/seclists/Discovery/Web-Content/raft-large-files.txt

Взламывающие Петли



/usr/share/seclists/Discovery/Web-Content/api/

Перечисление поддоменов	subdomains- top1million-20000.txt	/usr/share/seclists/Discovery/DNS/subdomains- top1million-20000.txt
Перечисление Vhost	subdomains- top1million-5000.txt	/usr/share/seclists/Discovery/DNS/subdomains- top1million-5000.txt
Размывание имени параметра	burp-parameter- names.txt	/usr/share/seclists/Discovery/Web- Content/burp-parameter-names.txt
Резервное копирование и файлы конфигурации	CommonBackdoors- PHP.fuzz.txt	/usr/share/seclists/Discovery/Web-Content/
Подбор пароля методом полного перебора (веб- формы)	rockyou.txt	/usr/share/wordlists/rockyou.txt

Быстрые советы по работе со списками слов

- Для ускорения работы начните с common.txt (4600 записей), а затем переходите к средним или большим спискам.
- Всегда добавляйте соответствующие расширения файлов с помощью `-x php,html,js,txt,bak,zip` — при сканировании необработанных каталогов файлы могут быть пропущены.
- Создайте список слов CeWL на основе целевого сайта для таргетированной рассылки:

4. Gobuster: все режимы и флаги — полный справочник

Gobuster распределяет свои функции по режимам. Сначала вы выбираете режим, а затем применяете флаги, характерные для этого режима.

Режимы Gobuster

Режим	Полное имя	Что он делает
реж	Каталог / файл	Перебор каталогов и файлов на веб-серверах. Самый распространенный режим.
dns	Поддомен DNS	Обнаруживает поддомены методом перебора DNS-записей.
vhost	Виртуальный хост	Находит виртуальные хосты на целевом сервере с помощью заголовка HTTP Host.
пушок	Размывание	Заменяет ключевое слово FUZZ в любом месте URL, заголовков или тела запроса.
s3	AWS S3	Перечисляет общедоступные корзины Amazon S3.
gcs	Облачное Хранилище Google	Перечисляет общедоступные корзины GCS.



Глобальные флаги (применяются ко всем режимам)

Отметить	Описание
-w, -wordlist	Путь к файлу со списком слов. Требуется для всех режимов.
-t, -нити	Количество параллельных потоков. Значение по умолчанию: 10. Увеличьте для повышения скорости (например, -t 50).
-o, -выходной сигнал	Записать результаты в файл: -o results.txt
-B, -тихо	Скрыть баннер и рекламу. Показать только результаты.
-z, -no-progress	Не показывать индикатор выполнения. Более чистый вывод для конвейера.
-v, -многословный	Подробный вывод. Отображает ошибки и дополнительную информацию.
-задержка	Делайте паузы между запросами, чтобы избежать ограничения скорости: -delay 100ms

Взламывающие Петли



ANSI. Полезно при ведении журнала в

-отлаживать

Печать отладочной информации. Полезно для устранения неполадок.

флаги режима dir

Отметить

Описание

<code>-u, -url</code>	Целевой URL. Обязательно. Например, <code>-u https://target.com</code>
<code>-x, -расширения</code>	Расширения файлов для добавления: <code>-x php,html,js,txt,bak</code> . Крайне важно для поиска файлов.
<code>-s, -status-codes</code>	Список кодов состояния, разделенных запятыми, которые считаются допустимыми: <code>-s 200,204,301,302,307,401,403</code>
<code>-b, -status-codes-blacklist</code>	Коды состояния, которые следует игнорировать: <code>-b 404,400</code> (отображать все, кроме этих)
<code>-l, -show-length</code>	Покажите длину основного текста ответа. Полезно для выявления интересных различий.
<code>-e, -расширенный</code>	Выводит полный URL (с протоколом и хостом), а не только путь.
<code>-r, -follow-redirect</code>	Следуйте HTTP-перенаправлениям и сообщайте конечный URL.

Взламывающие Петли



/ сертификата TLS/SSL. Использовать для
анными сертификатами.

-H, -заголовки	Добавьте пользовательские HTTP-заголовки: -H 'Authorization: Bearer token123'
-c, -файлыcookie	Добавить файлы cookie: -c 'session=abc123; role=admin'
-U, -имяпользователя	Имя пользователя для базовой аутентификации: -U admin
-P, -пароль	Пароль для базовой аутентификации: -P password123
-прокси -сервер	Направьте трафик через прокси: -proxy http://127.0.0.1:8080 (Burp Suite)
-тайм-аут	Тайм-аут запроса: -timeout 10s
-случайный агент	Используйте случайный заголовок User-Agent для каждого запроса. Это поможет избежать обнаружения.
-исключить-длина	Исключить ответы с определенной длиной тела: -exclude-length 1234

флаги режима dns

Отметить	Описание

Взламывающие Петли



rget.com

нные IP-адреса вместе с обнаруженными
поддоменами.

<code>-c, -show-cname</code>	Показать записи CNAME для обнаруженных поддоменов.
<code>-подстановочный знак</code>	Принудительное продолжение работы даже при обнаружении подстановочного DNS.
<code>-распознаватель</code>	Используйте собственный DNS-преобразователь: <code>-resolver 8.8.8.8</code> (Google DNS)

флаги режима vhost

Отметить	Описание
<code>-u, -url</code>	Целевой URL (используйте IP-адрес, а не имя хоста): <code>-u http://10.10.10.10</code>
<code>-добавить-домен</code>	Добавьте базовый домен к каждой записи в списке слов: <code>api</code> → <code>api.target.com</code>
<code>-исключить-длина</code>	Исключить ответы, соответствующие определенной длине (отфильтровать ответ виртуального хоста по умолчанию)

5. Gobuster: примеры команд из реальной жизни

Взламывающие Петли

[illegible]

Взламывающие Петли



```
ommon.txt -delay 500ms -t 5
```

```
ommon.txt -q -o results.txt
```

DNS Subdomain Enumeration

```
# Basic subdomain enumeration

gobuster dns -d target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt

# Show resolved IPs

gobuster dns -d target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \

    -i -t 100

# Show CNAME records too

gobuster dns -d target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \

    -i -c

# Force continue with wildcard DNS

gobuster dns -d target.com -w subdomains.txt -wildcard

# Use custom DNS resolver

gobuster dns -d target.com -w subdomains.txt -resolver 8.8.8.8 -i
```

Virtual Host (vhost) Enumeration

```
# Basic vhost enumeration (use IP, not hostname)

gobuster vhost -u http://10.10.10.10 \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

# Append domain to wordlist entries (api → api.target.com)

gobuster vhost -u http://10.10.10.10 \
```

Взламывающие Петли



```
subdomains-top1million-5000.txt \
```

```
the common page length)
```

```
# First run without filter, note the default response length, then:

gobuster vhost -u http://10.10.10.10 \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \

    -append-domain -exclude-length 612

# With HTTPS and skip TLS verification

gobuster vhost -u https://target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \

    -append-domain -k -t 50
```

Gobuster fuzz Mode

```
# Fuzz a URL parameter value

gobuster fuzz -u 'https://target.com/page.php?id=FUZZ' \

    -w /usr/share/seclists/Fuzzing/fuzz-Bo0oM.txt

# Find hidden parameters

gobuster fuzz -u 'https://target.com/api/endpoint?FUZZ=test' \

    -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt

# Fuzz with status code filter

gobuster fuzz -u 'https://target.com/FUZZ' -w wordlist.txt \

    -exclude-length 0 -b 404,400
```

Gobuster doesn't do recursion — here's the workaround

Gobuster does not automatically scan discovered directories. When you find /admin, you need to run a separate scan: `gobuster dir -u https://target.com/admin -w common.txt -x php`. Many pentesters write a simple loop or use ffuf with `-recursion` instead for deeper exploration.

Взламывающие
Петли

ntax

≡

word FUZZ anywhere in your request — in the cookies, POST body, or even the Host header — and ffuf replaces it with each entry from your wordlist. This makes it far more versatile than Gobuster for anything beyond simple path busting.

Basic syntax

```
ffuf -u <URL_with_FUZZ> -w <wordlist> [options]

# The three things you almost always need:

# -u      : Target URL containing FUZZ placeholder

# -w      : Wordlist path

# -fc     : Filter out unwanted status codes (e.g., -fc 404)
```

Core ffuf Flags — Complete Reference

Flag	Description
-u	Target URL with FUZZ keyword: -u https://target.com/FUZZ
-w	Wordlist: -w /path/to/wordlist.txt or -w wordlist.txt:KEYWORD for named keywords
-H	Custom header (can use multiple times): -H 'Authorization: Bearer token'
-X	HTTP method: -X POST (default: GET)

Взламывающие Петли



sername=admin&password=FUZZ'

c123; FUZZ=test'

-t	Threads (default: 40): -t 100 for speed, -t 5 for stealth
-p	Delay between requests: -p 0.1 (100ms), -p 1 (1 second)
-r	Follow redirects
-s	Silent mode — suppress progress bar and info messages
-v	Verbose — show full URLs in output
-o	Output file: -o results.json
-of	Output format: -of json, -of csv, -of html, -of md (markdown)
-k	Skip TLS certificate verification
-x	HTTP proxy: -x http://127.0.0.1:8080 (Burp Suite)
-recursion	Enable recursive scanning of found directories
-recursion-depth	Max recursion depth: -recursion-depth 3

Взламывающие Петли



a-separated): -e php,html,js,bak

-maxtime 300

-rate	Max requests per second: -rate 100
-request	Load a saved raw HTTP request file: -request req.txt
-request-proto	Protocol for -request mode: -request-proto https

7. ffuf: Filters & Matchers — The Key to Useful Output

Without filters, ffuf floods you with thousands of results, most of them 404s or identical default pages. Filters and matchers are what turn raw noise into actionable findings. This is the skill that separates beginners from effective fuzzers.

Filter Flags (fc, fs, fw, fl, ft — exclude matching responses)

Flag	Filters by	Example
-fc	HTTP status code(s)	-fc 404 or -fc 404,400,403
-fs	Response size (bytes)	-fs 1234 or -fs 100-200
-fw	Word count in response	-fw 10 (exclude responses with exactly 10 words)

Взламывающие Петли



		-fl 1 (exclude single-line responses)
		-ft 500 (exclude responses faster than 500ms)
-fr	Regex match in response body	-fr "Not Found" (exclude responses containing this text)

Matcher Flags (mc, ms, mw, ml, mt, mr — show only matching responses)

Flag	Matches by	Example
-mc	HTTP status code(s)	-mc 200 or -mc 200,301,302
-ms	Response size (bytes)	-ms 5000 (only show responses of exactly 5000 bytes)
-mw	Word count in response	-mw 50 (only responses with 50 words)
-ml	Line count in response	-ml 25
-mr	Regex match in response body	-mr "admin dashboard welcome"
-mt	Response time (ms)	-mt 5000 (only show responses taking >5 seconds — blind injection)

2. Note the size/word count of the 404 or default page. Use `-fs <that_size>` or `-fw <that_wordcount>` to filter it out.
3. If all 404s return the same 3-word response, use `-fw 3`. If they're all 1234 bytes, use `-fs 1234`.
4. Use `-mc 200,301,302` to show only clearly successful responses. Use `-fc 404` to drop only 404s and show everything else (including 403s, which are often interesting).

403 Forbidden is NOT a dead end

A 403 response means the path exists but access is restricted. Always record 403 results — they indicate sensitive functionality worth attempting to bypass. Common bypass techniques include adding a trailing slash (/admin/), modifying the HTTP method, or injecting headers like X-Original-URL: /admin.

8. ffuf: All Fuzzing Modes with Full Commands

Directory & File Fuzzing

```
# Basic directory fuzzing

ffuf -u https://target.com/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt

# Filter out 404s (essential for clean output)

ffuf -u https://target.com/FUZZ -w common.txt -fc 404

# Show only 200 and redirect responses

ffuf -u https://target.com/FUZZ -w common.txt -mc 200,301,302

# With file extensions
```

Взламывающие Петли



```
ffuf -u https://target.com/FUZZ \  
  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \  
  
-e .php,.html,.js,.bak,.zip,.conf \  
  
-fc 404 -t 50 -o results.json -of json  
  
# Recursive scanning  
  
ffuf -u https://target.com/FUZZ -w common.txt -fc 404 \  
  
-recursion -recursion-depth 3  
  
# With Burp Suite proxy for inspection  
  
ffuf -u https://target.com/FUZZ -w common.txt -fc 404 \  
  
-x http://127.0.0.1:8080 -k  
  
# Filter by response size (useful when 404 page has consistent size)  
  
ffuf -u https://target.com/FUZZ -w common.txt -fs 1234  
  
# Filter by word count  
  
ffuf -u https://target.com/FUZZ -w common.txt -fw 10
```

GET Parameter Fuzzing

```
# Find hidden GET parameters (parameter name is FUZZ)  
  
ffuf -u 'https://target.com/page.php?FUZZ=test' \  
  
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \  
  
-fs 1234  
  
# Fuzz the value of a known parameter  
  
ffuf -u 'https://target.com/page.php?id=FUZZ' \  
  
-w /usr/share/seclists/Fuzzing/fuzz-Bo0oM.txt \  
  
-fc 404 -mc 200,500  
  
# LFI testing - fuzz path traversal payloads
```

Взламывающие Петли

le=FUZZ' \
I-Jhaddix.txt \
≡

```
# IDOR testing - fuzz numeric IDs

ffuf -u 'https://target.com/api/user/FUZZ' \  
-w /usr/share/seclists/Fuzzing/Digits/1-1000.txt \  
-fc 404 -mc 200
```

POST Data Fuzzing

```
# Brute-force a login form password

ffuf -u https://target.com/login -X POST \  
-d 'username=admin&password=FUZZ' \  
-w /usr/share/wordlists/rockyou.txt \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
-fc 401 -mc 200,302

# Username enumeration (fuzz the username field)

ffuf -u https://target.com/login -X POST \  
-d 'username=FUZZ&password=Password123' \  
-w /usr/share/seclists/Usernames/top-usernames-shortlist.txt \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
-fs 1234

# JSON POST body fuzzing

ffuf -u https://api.target.com/login -X POST \  
-d '{"username":"admin","password":"FUZZ"}' \  
-w /usr/share/wordlists/rockyou.txt \  
-H 'Content-Type: application/json' \  
-fc 401

# Fuzz a hidden JSON parameter name
```

Взламывающие Петли



```
-X POST \
```

```
Content/burp-parameter-names.txt \
```

```
-H 'Content-Type: application/json' \
```

```
-fs 0 -mc 200
```

Subdomain Fuzzing

```
# Basic subdomain brute force

ffuf -u https://FUZZ.target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \

    -fc 404 -mc 200,301,302,403

# With size filter (exclude default 'domain not found' response)

ffuf -u https://FUZZ.target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \

    -fs 1234

# Verbose output to see full URLs

ffuf -u https://FUZZ.target.com \

    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \

    -fc 404 -v
```

Virtual Host Fuzzing

```
# Fuzz the Host header to find virtual hosts

# First: do a baseline request to see the default response size

curl -s http://10.10.10.10 | wc -c

# Then filter that size out:

ffuf -u http://10.10.10.10 \

    -H 'Host: FUZZ.target.com' \
```

Взламывающие Петли



```
ffuf -u https://10.10.10.10 \  
  
-H 'Host: FUZZ.target.com' \  
  
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \  
  
-fs 4242 -k
```

Header Fuzzing

```
# Fuzz a custom header value (e.g., X-Forwarded-For bypass)  
  
ffuf -u https://target.com/admin \  
  
-H 'X-Forwarded-For: FUZZ' \  
  
-w /usr/share/seclists/Fuzzing/IPv4-Addresses.txt \  
  
-mc 200  
  
# Fuzz the User-Agent header  
  
ffuf -u https://target.com \  
  
-H 'User-Agent: FUZZ' \  
  
-w /usr/share/seclists/Fuzzing/User-Agents/user_agents.txt \  
  
-fs 1234  
  
# Fuzz header name (find interesting headers the server accepts)  
  
ffuf -u https://target.com/api/v1/users \  
  
-H 'FUZZ: 127.0.0.1' \  
  
-w /usr/share/seclists/Fuzzing/http-request-headers/http-request-headers-common.txt \  
  
-fs 0 -mc 200
```

Cookie Fuzzing

```
# Fuzz a cookie value (e.g., role-based access control bypass)
```



```
s/generics.txt \
```

```
-mc 200 -fc 403

# Brute-force a session token

ffuf -u https://target.com/dashboard \

    -b 'session=FUZZ' \

    -w sessions_wordlist.txt \

    -mc 200
```

9. ffuf: Advanced Techniques

Multiple Wordlists (Named Keywords)

Replace FUZZ with custom keywords when you need to fuzz multiple positions simultaneously:

```
# Two-position fuzzing: username AND password (pitchfork-style)

ffuf -u https://target.com/login -X POST \

    -d 'username=USER&password=PASS' \

    -w usernames.txt:USER \

    -w passwords.txt:PASS \

    -H 'Content-Type: application/x-www-form-urlencoded' \

    -mode pitchfork \

    -fc 401

# Cluster bomb mode (every combination - use carefully)

ffuf -u https://target.com/login -X POST \

    -d 'username=USER&password=PASS' \

    -w usernames.txt:USER \

    -w passwords.txt:PASS \
```


Взламывающие Петли



```
# Multiple positions in URL

ffuf -u 'https://target.com/api/W1/resource/W2' \

    -w versions.txt:W1 \

    -w endpoints.txt:W2 \

    -mode pitchfork -fc 404
```

Using a Raw HTTP Request File

Capture a request in Burp Suite, save it, then feed it directly to ffuf — FUZZ goes anywhere in the saved request:

```
# Save request from Burp: right-click → Save item → request.txt

# Place FUZZ in the saved request file where you want fuzzing to occur

# Example request.txt content:

# POST /login HTTP/1.1

# Host: target.com

# Content-Type: application/x-www-form-urlencoded

#

# username=admin&password=FUZZ

# Run ffuf with the saved request

ffuf -request request.txt -request-proto https \

    -w /usr/share/wordlists/rockyou.txt -fc 401

# Pitchfork with two placeholders in request file

ffuf -request request.txt -request-proto https \

    -w usernames.txt:USER -w passwords.txt:PASS \

    -mode pitchfork -fc 401
```



```
ffuf -u https://api.target.com/v1/FUZZ \

-w /usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt \

-H 'Authorization: Bearer <token>' \

-fc 404 -mc 200,201,400,403

# Fuzz API version

ffuf -u https://api.target.com/FUZZ/users \

-w /usr/share/seclists/Discovery/Web-Content/api/api-seen-in-wild.txt \

-fc 404

# Find undocumented API routes

ffuf -u https://api.target.com/api/FUZZ \

-w /usr/share/seclists/Discovery/Web-Content/raft-large-words.txt \

-H 'Content-Type: application/json' \

-H 'Authorization: Bearer <token>' \

-fc 404 -t 30
```

Blind SQL Injection Detection via Response Time

```
# Use -mt to find parameters causing time delays (blind SQLi)

ffuf -u 'https://target.com/search.php?q=FUZZ' \

-w /usr/share/seclists/Fuzzing/SQLi/Generic-SQLi.txt \

-fc 404 \

-mt 5000

# -mt 5000 = only show responses that take >5000ms (5 seconds)

# A time delay suggests time-based blind SQL injection
```

10. Gobuster vs ffuf: Side-by-Side Comparison

Взламывающие Петли

| a real engagement:



Why

Quick directory bust on a new target	Gobuster	Less to type, fast, simple output
DNS subdomain enumeration	Gobuster dns	Better DNS-specific options (-i, -c, -resolver)
Virtual host discovery	Either	Both work well; ffuf's -fs filter is easier to calibrate
POST form login brute force	ffuf	Gobuster can't fuzz POST bodies; ffuf handles this natively
GET parameter discovery	ffuf	FUZZ can be placed in the parameter name or value position
API endpoint fuzzing	ffuf	Better header control, JSON body support, named keywords
Recursive directory scanning	ffuf	Gobuster has no recursion; ffuf -recursion handles it

Взламывающие Петли



		ffuf can place FUZZ in any header or cookie value
Blind injection timing attacks	ffuf	-mt flag detects time-delayed responses
Piping output for scripting	Gobuster -q	Cleaner bare output; ffuf JSON is richer but needs parsing
Multi-position fuzzing (pitchfork)	ffuf	Named keywords + -mode pitchfork/clusterbomb
Filtering false positives precisely	ffuf	fc/fs/fw/fl/fr/ft give much finer control than Gobuster

Other tools worth knowing

Tool	Standout feature	Install
Feroxbuster	Recursive scanning with Rust speed. Best for deep recursive enumeration.	cargo install feroxbuster
Dirsearch	Python-based, great default config, built-in extension handling.	pip install dirsearch

Взламывающие Петли



rful

ring.

pip install wfuzz

dirb

Pre-installed on Kali, simple,
good for beginners. Very slow.

sudo apt install dirb

11. The Web Enumeration Workflow: Where These Tools Fit

Directory and parameter fuzzing has a specific place in the web application pentest methodology. Using it at the right time — and in the right order — is what makes the difference between finding nothing and finding everything.

Phase 1: Passive recon (before you fuzz)

- Browse the application manually with Burp Suite's proxy running. Let the site map build organically. See the [Burp Suite Cheat Sheet](#) on hackingloops.com for the full setup guide.
- Note all technologies visible in headers (X-Powered-By, Server), response bodies, and source comments.
- Check robots.txt and sitemap.xml for hidden paths the site itself advertises.
- View page source for commented-out links, API endpoints, JavaScript file references.

Phase 2: Initial directory sweep (Gobuster first)

- Run Gobuster with `common.txt` and common extensions. This takes seconds and catches 70% of findings.
- Review all 301/302 redirects — follow them manually.
- Record all 403 responses — these indicate protected but existing resources.
- Note all interesting directories for deeper scanning in Phase 3.

≡ ith larger wordlists and recursive scanning.

pages: ffuf -u 'https://target.com/page.php?

```
FUZZ=test' -w burp-parameter-names.txt -fs 1234.
```

- Enumerate subdomains and virtual hosts if the target scope includes them.
- Test API endpoints discovered from Burp's JS Link Finder extension.

Phase 4: Targeted fuzzing (exploitation support)

- Found a login form? Use ffuf to brute force or enumerate usernames.
- Found a file upload endpoint? Fuzz file extensions to find accepted types.
- Found an ID parameter? Fuzz numeric values to test for IDOR.
- Found an interesting parameter? Fuzz the value with injection payload lists.
- Pass anything that needs deeper testing to Burp Suite Repeater for manual analysis. See the [Burp Suite Cheat Sheet](#) →
- For any SQL injection candidates found, pass to SQLmap. See the [SQLmap Cheat Sheet](#) →

12. Common Mistakes & Pro Tips

Mistakes That Cost Time

Mistake 1: Not using extensions in dir mode

Running gobuster dir without -x php,html,js,bak will completely miss files. A hidden admin panel called /admin/login.php won't appear if you're only looking for directories. Always add extensions for web targets.

Mistake 2: Not filtering ffuf output

Running ffuf without any -fc or -fs filter gives you thousands of results — most useless. Always do a baseline request first, note the default error response size, then filter it out with -fs. If you don't know what to filter, start with -fc 404 at minimum.

Mistake 3: Using too many threads on a production target

Running 100 threads against a small production server can crash it or trigger alerts. For targets with rate limiting or WAFs, use -t 10 and -delay 200ms. Save the high



g — even non-existent pages — with a custom error message. A 200 response doesn't mean you found something. Check the response size and content, not just the status code. Use -fs to filter responses of the same size as the error page.

Mistake 5: Missing the /api/ prefix

Modern web apps often have their API under /api/v1/ or /api/v2/ — sometimes both. Always run a separate scan with the API-specific wordlists from SecLists:
`/usr/share/seclists/Discovery/Web-Content/api/`

Pro Tips

Tip 1: Always check 403 responses

Filter to show 403s alongside 200s with `gobuster dir -b 404` or `ffuf -mc 200,301,302,403`. Found /admin returning 403? Try: appending a trailing slash (/admin/), using a different HTTP method, adding X-Forwarded-For: 127.0.0.1 header, or trying /ADMIN, /%2fadmin, or ../admin/.

Tip 2: Fuzz after finding each new directory

Gobuster doesn't recurse. When you find /backup, immediately run: `gobuster dir -u https://target.com/backup -w common.txt -x php,zip,bak`. Or use ffuf with -recursion so it handles this automatically.

Tip 3: Use the -e flag wisely — backup and config files are goldmines

Always include `.bak`, `.old`, `.zip`, `.tar.gz`, `.swp`, `.config`, `.conf`, `.env` in your extension list. Developers often leave backup copies of configuration files — `.env` files especially can contain database credentials, API keys, and secret tokens.

Tip 4: Use -o to save output, always

Fuzzing output scrolls fast and you'll miss results. Save everything with `-o output.txt` (Gobuster) or `-o output.json -of json` (ffuf). JSON output from ffuf is especially useful as it records every request's status, size, and words for later analysis.

Tip 5: Slow down against WAF-protected targets

If a target has a Web Application Firewall, fast fuzzing will get you blocked instantly. Use `-t 5 -delay 500ms` (Gobuster) or `-t 5 -p 0.5 -rate 10` (ffuf) to stay under the radar.

gent strings.

Взламывающие Петли



the quick first sweep → ffuf for deep

parameter and post-body fuzzing → Burp Suite for manual analysis of interesting findings → SQLmap or other exploit tools where injection is suspected. Each tool in the chain narrows the attack surface for the next.

13. Quick Reference Summary

The condensed version for bookmarking.

Gobuster quick commands

Task	Command
Basic dir scan	<code>gobuster dir -u https://target.com -w common.txt</code>
Dir scan + extensions	<code>gobuster dir -u https://target.com -w common.txt -x php,html,js,bak</code>
Show all except 404	<code>gobuster dir -u https://target.com -w common.txt -b 404</code>
DNS subdomain enum	<code>gobuster dns -d target.com -w subdomains-top1million-20000.txt -i</code>
Vhost discovery	<code>gobuster vhost -u http://10.10.10.10 -w subdomains-top1million-5000.txt -append-domain</code>

Взламывающие Петли



```
curl -H 'Host: target.com' -w common.txt -
```

```
-H 'Authorization: Bearer TOKEN'
```

With cookie	<pre>gobuster dir -u https://target.com/admin -w common.txt -c 'session=abc123'</pre>
Via Burp proxy	<pre>gobuster dir -u https://target.com -w common.txt -proxy http://127.0.0.1:8080 -k</pre>

ffuf quick commands

Task	Command
Basic dir fuzzing	<pre>ffuf -u https://target.com/FUZZ -w common.txt -fc 404</pre>
Dir + extensions	<pre>ffuf -u https://target.com/FUZZ -w common.txt -e .php,.html,.bak -fc 404</pre>
Recursive scan	<pre>ffuf -u https://target.com/FUZZ -w common.txt -fc 404 -recursion -recursion-depth 3</pre>
GET param discovery	<pre>ffuf -u 'https://target.com/page.php?FUZZ=test' -w burp-parameter-names.txt -fs 1234</pre>
POST login brute force	<pre>ffuf -u https://target.com/login -X POST -d 'user=admin&pass=FUZZ' -w rockyou.txt -fc 401</pre>

Взламывающие Петли

```
10.10.10.10 -H 'Host:
' -w subdomains.txt -fs 4242
```

Subdomain fuzzing	<pre>ffuf -u https://FUZZ.target.com -w subdomains- top1million-5000.txt -fc 404</pre>
Pitchfork (user+pass)	<pre>ffuf -u https://target.com/login -X POST -d 'user=USER&pass=PASS' -w u.txt:USER -w p.txt:PASS -mode pitchfork -fc 401</pre>
Raw request file	<pre>ffuf -request request.txt -request-proto https -w wordlist.txt -fc 404</pre>
Save JSON output	<pre>ffuf -u https://target.com/FUZZ -w common.txt -fc 404 -o results.json -of json</pre>

Essential wordlist paths (Kali Linux)

Use case	Path
Quick dir bust	<pre>/usr/share/seclists/Discovery/Web- Content/common.txt</pre>
Thorough dir bust	<pre>/usr/share/wordlists/dirbuster/directory-list-2.3- medium.txt</pre>

Взламывающие Петли

ists/Discovery/Web-Content/raft-



API endpoints	<code>/usr/share/seclists/Discovery/Web-Content/api/api-endpoints.txt</code>
Subdomains (fast)	<code>/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt</code>
Subdomains (thorough)	<code>/usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt</code>
Parameter names	<code>/usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt</code>
Password brute force	<code>/usr/share/wordlists/rockyou.txt</code>

What to learn next

These tools complete the web app hacking recon and fuzzing phase:

- **Burp Suite Cheat Sheet** — intercept and manually analyse everything your fuzzing discovers. Internal link →
- **SQLmap Cheat Sheet** — when fuzzing finds SQLi candidates, SQLmap does the exploitation. Internal link →
- **Nikto Cheat Sheet** — complement directory busting with web server vulnerability scanning. Internal link →



This was created for those who want to obtain a lucrative career in cybersecurity. Get this free guide delivered to your email that will show you step-by-step how to get into cybersecurity.

Quick Link

[Home](#)

[About Us](#)

[Membership](#)

[Blog](#)

[Contact Us](#)

Contact Detail

500 Westover Dr #8208, Sanford, NC 27330

hackingloopsteam@hackingloops.com